

Pathtracer - Raynaldo

Relatório de Visualização e Iluminação

Francisco Ferreira
PG55942

Ivan Ribeiro
PG55950

Júlio Pinto
PG57883

Introdução

Este projeto teve como objetivo desenvolver um *pathtracer*. Ao contrário de seguir o exemplo fornecido pelo professor, optámos por desenvolver uma implementação completamente nova em *Rust*, aproveitando as características e vantagens desta linguagem.

Funcionalidades

As funcionalidades implementadas neste trabalho incluem:

- Multithreading
- Carregamento de cenas de OBJ
- Janela interativa com pathtracer progressivo
- Tonemapping
- Estrutura de aceleração (*BVH*)
- *Participating Media*

Todas as funcionalidades foram implementadas com sucesso, exceto a funcionalidade de *Participating Media*, que foi removida do âmbito do projeto. Relativamente ao carregamento de ficheiros *OBJ*, a implementação atual suporta apenas a geometria, não incluindo o carregamento de materiais associados.

Janela Interativa com Pathtracer Progressivo e Multithreading

Como funcionalidade principal do trabalho, foi implementada uma janela interativa com pathtracer progressivo.

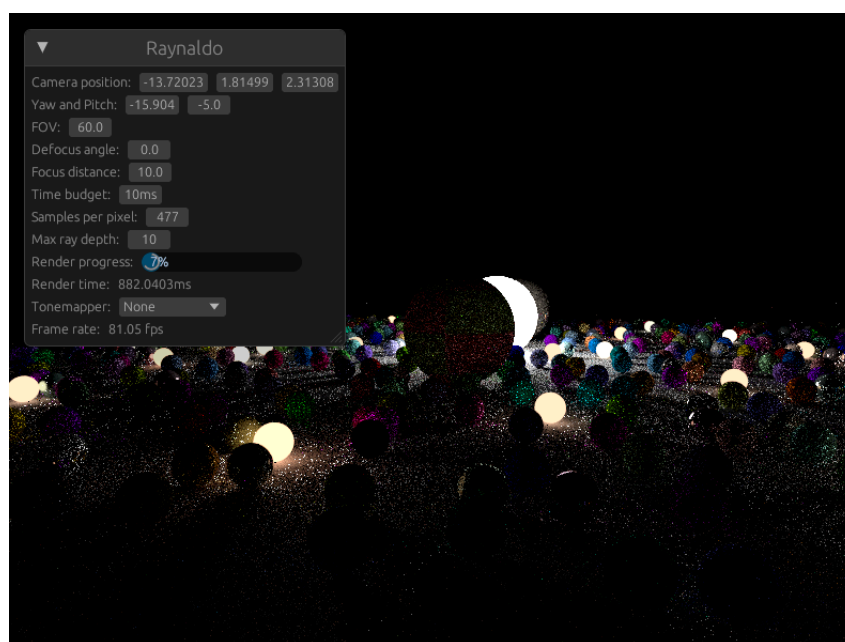


Figura 1: Interface do pathtracer

Arquitetura da Aplicação

A aplicação é estruturada em torno de um *loop* principal que integra três componentes fundamentais: o sistema de janelas *winit*, o *renderer* na *GPU* via *wgpu*, e a interface gráfica *egui*.

O sistema *winit* gere todos os eventos da janela (redimensionamento, *input* do utilizador, etc.), enquanto *wgpu* fornece uma abstração moderna sobre *APIs* gráficas (*Vulkan*, *Metal*, *DirectX 12*) para transferência eficiente de dados para a *GPU*. A biblioteca *egui* oferece uma interface de modo imediato que é renderizada em cima da textura de *output* do *pathtracer*.

Sistema de Renderização Progressiva

O núcleo do sistema de renderização progressiva reside na estrutura *RenderState*, que mantém um *canvas* de acumulação onde cada pixel armazena um tuplo (*r_sum*, *g_sum*, *b_sum*, *sample_count*). A cada *sample* renderizado é somado o valor do pixel a cada canal de cor, e o contador de amostras é incrementado.

Aquando de uma chamada de renderização, o sistema começa a renderizar os píxeis da imagem. Este processamento é feito em pacotes de 10.000 píxeis, onde são renderizados em paralelo via uma biblioteca de *threads* (*rayon*). Isto é repetido até que o sistema atinja o número de amostras por pixel desejado ou que tenha excedido o tempo de *CPU* dedicado por *frame* ao *pathtracing*. Este valor de tempo de *CPU* é configurável na *UI* do *egui*. Quanto maior for este valor, menos responsiva será a interface.

Quando o utilizador altera parâmetros de renderização (como a posição da câmara ou definições de renderização), o *canvas* de acumulação é restaurado. Para melhorar qualidade visual durante a renderização progressiva, o sistema evita renderizar os píxeis sempre na mesma ordem.

Para isso, o sistema usa *arrays* de índices precalculados que definem ordens de renderização aleatórias. Durante a renderização, o sistema consulta um desses *arrays* para determinar qual pixel renderizar a cada iteração sequencial. O sistema mantém 5 *arrays* destes diferentes, e alterna entre eles sempre que os parâmetros de renderização são alterados.

Isto dá a impressão de que a ordem de renderização é aleatória, no entanto, mantendo-se altamente eficiente e não comprometendo acessos concorrentes a píxeis.

```
while !state.render_state.is_finished(state.samples_per_pixel)
    && instant.elapsed() < Duration::from_millis(state.time_budget_ms)
{
    // Processamento paralelo de lote de pixels
    state.render_state.get_current_pixel_order()
        [state.render_state.current_render_pixel..end]
        .par_iter()
        .for_each(|&index_in_buffer| {
            let x = index_in_buffer as u32 % width;
            let y = index_in_buffer as u32 / width;
            let color = state.scene.render_sample(x, y, state.max_ray_depth);

            state.render_state.canvas.increment_pixel(x, y, color);
        });
}
```

Depois disto, o *canvas* é convertido para um *array* de *bytes*, onde é feito o *tonemapping* e a divisão das cores pelo número de amostras feitas nesse píxel, e então é transferido para a textura que reside na *GPU*.

A *GPU* irá renderizar a textura para o ecrã de acordo com o *shader* que está a ser utilizado.

```
@group(0) @binding(0) var tex: texture_2d<f32>;
@group(0) @binding(1) var tex_sampler: sampler;

struct VertexOutput {
    @builtin(position) position: vec4<f32>,
    @location(0) uv: vec2<f32>,
};

@vertex
fn vs_main(@builtin(vertex_index) vertex_index: u32) -> VertexOutput {
    var out: VertexOutput;

    // Gerar triângulo de acordo com o índice do vértice
    let x: f32 = f32((vertex_index << 1u) & 2u) * 2.0 - 1.0;
    let y: f32 = f32(vertex_index & 2u) * 2.0 - 1.0;
    out.position = vec4<f32>(x, y, 0.0, 1.0);

    out.uv = (vec2<f32>(x, y) + vec2<f32>(1.0)) * 0.5;
    return out;
}

@fragment
fn fs_main(input: VertexOutput) -> @location(0) vec4<f32> {
    return textureSample(tex, tex_sampler, input.uv);
}
```

O *shader* é composto por um *vertex shader* que gera um triângulo de acordo com o índice do vértice, e um *fragment shader* que amostra a textura no píxel correspondente.

Sistema de Interação

Como controlo da câmara foi implementado um sistema de controlo *FPV*, onde o utilizador pode mover a câmara com as teclas *WASD* para movimento horizontal e frontal, usando espaço e *shift* para subir e descer respectivamente. Ao premir o botão esquerdo do rato, é possível rodar a câmara em torno do seu eixo através de movimentos de rotação *yaw* e *pitch*.

Com a interface do *egui* é possível visualizar e/ou modificar os seguintes parâmetros:

- Posição da câmara
- *Yaw* e *Pitch* da câmara
- *FOV* da câmara
- Ângulo de desfoque da câmara
- Distância de foco da câmara
- Orçamento de tempo por frame
- Samples por píxel
- Profundidade máxima de raios
- Progresso da renderização
- Tonemapping
- *Framerate*
- Tempo de renderização

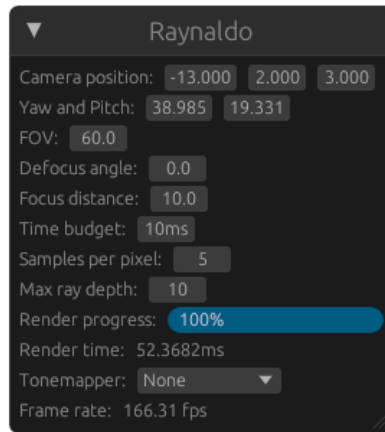


Figura 2: Interface do pathtracer

Carregamento da cena

As cenas do *pathtracer* podem ser carregadas via um ficheiro *TOML*, que define as propriedades da câmara, o ambiente, as geometrias e os materiais.

Estrutura do Ficheiro de Configuração

Cada ficheiro de cena está organizado hierarquicamente em três secções principais que definem completamente o estado inicial da renderização.

A configuração da câmara, especificada através da secção `[camera]`, estabelece a posição inicial do observador no espaço tridimensional, a orientação através de ângulos *yaw* e *pitch*, o *field of view* que determina a abertura angular da perspectiva, e parâmetros avançados de profundidade de campo incluindo *focus_distance* e *defocus_angle* para simulação de efeitos de *bokeh* e desfoque.

A iluminação ambiental é controlada através da secção `[environment]`, que permite especificar o comportamento da luz de fundo quando raios não intersejam nenhuma geometria da cena, suportando tanto cores sólidas uniformes através de `type = "color"` com uma especificação da cor, como gradientes procedurais semelhantes ao céu via `type = "sky"`.

As geometrias da cena são definidas através de múltiplas secções `[[geometry]]`, cada uma especificando um objeto tridimensional incluindo o seu tipo geométrico primitivo, propriedades de material físico, e parâmetros específicos como posição, orientação e escala. Também define as propriedades do material.

```
[camera]
position = [1.85, 1.85, -4.0]
yaw = 90.0
pitch = 0.0
fov = 60.0
focus_distance = 0.1
defocus_angle = 0.0
```

```
[environment]
type = "sky"
```

```
[[geometry]]
```

```

type = "sphere"
center = [0.0, 0.0, 0.0]
radius = 1.0
material = "lambertian"
texture = "solid"
color = [0.7, 0.3, 0.3, 1.0]

```

A deserialização deste ficheiro é feita via biblioteca *serde* que simplifica completamente este trabalho, tendo que apenas definir as *structs* que representam os dados do ficheiro.

Tipos de Geometria Suportados

O sistema suporta quatro tipos principais de geometria primitiva, cada um configurado através de parâmetros específicos no ficheiro *TOML*.

Esferas são definidas por dois parâmetros essenciais: *center* especifica a posição central da esfera no espaço tridimensional através de coordenadas [*x*, *y*, *z*], e *radius* define o raio escalar que determina o tamanho da esfera.

```

[[geometry]]
type = "sphere"
center = [0.87, 0.5, 0.43]
radius = 0.5
material = "dielectric"
refractive_index = 1.5

```

Quads representam superfícies planares retangulares configuradas através de três vetores: *origin* define o ponto de partida da superfície, *u* especifica o primeiro vetor de orientação que determina uma das dimensões do retângulo, e *v* define o segundo vetor perpendicular que completa a definição da superfície.

```

[[geometry]]
type = "quad"
origin = [0.0, 0.0, 3.7]
u = [3.7, 0.0, 0.0]
v = [0.0, 0.0, -3.7]
material = "lambertian"
texture = "solid"
color = [0.73, 0.73, 0.73, 1.0]

```

Malhas de Triângulos podem ser carregadas de ficheiros *OBJ* externos ou definidas implicitamente. Para modelos *OBJ*, a configuração requer *mesh_type* = "*obj_file*" e *path* especificando o caminho para o ficheiro do modelo. Para definição implícita, utiliza-se *mesh_type* = "*implicit*" com arrays *verts* contendo coordenadas de vértices e índices.

```

[[geometry]]
type = "triangle_mesh"
mesh_type = "obj_file"
path = "assets/dragon8k.obj"
material = "metal"
albedo = [0.9, 0.8, 0.6, 1.0]
fuzziness = 0.1

```

Caixas Orientadas são configuradas através de quatro vetores: *origin* define o ponto de partida da caixa, enquanto *u*, *v* e *w* especificam os três vetores de orientação que determinam as dimensões nas três direções principais.

```
[[geometry]]
type = "box"
origin = [1.76, 0.0, 1.96]
u = [1.046, 0.0, -0.340]
v = [0.0, 2.2, 0.0]
w = [0.340, 0.0, 1.046]
material = "lambertian"
texture = "solid"
color = [0.73, 0.73, 0.73, 1.0]
```

Sistema de Materiais

O sistema suporta quatro tipos de materiais, cada um configurado através de parâmetros específicos no ficheiro *TOML*.

Materiais Lambertianos representam superfícies difusas e suportam três tipos de textura.

Para texturas sólidas usa-se `texture = "solid"` com `color = [r, g, b, a]` especificando a cor *RGBA*. Para padrões de xadrez configura-se `texture = "checker"` com `color1` e `color2` definindo as duas cores alternantes e `scale` controlando o tamanho de cada quadrado do padrão. Para texturas de imagem utiliza-se `texture = "image"` com `image` especificando o caminho para o ficheiro de textura.

```
# Cor sólida
material = "lambertian"
texture = "solid"
color = [0.7, 0.3, 0.3, 1.0]

# Padrão de xadrez
material = "lambertian"
texture = "checker"
color1 = [0.2, 0.3, 0.1, 1.0]
color2 = [0.9, 0.9, 0.9, 1.0]
scale = 0.32

# Textura de imagem
material = "lambertian"
texture = "image"
image = "assets/img.png"
```

Materiais Metálicos são configurados através de `material = "metal"` com dois parâmetros: `albedo` define a cor base do metal em formato *RGBA*, e `fuzziness` controla a rugosidade da superfície com valores entre 0.0 (perfeitamente liso) e 1.0 (muito rugoso).

```
material = "metal"
albedo = [0.8, 0.8, 0.9, 1.0]
fuzziness = 0.1
```

Materiais Dielétricos representam materiais transparentes e requerem apenas `material = "dielectric"` e `refractive_index` que especifica o índice de refração do material (por exemplo, 1.5 para vidro, 1.33 para água).

```
material = "dielectric"
refractive_index = 1.5
```

Materiais Emissivos funcionam como fontes de luz e são configurados através de `material = "emissive"` com `color` definindo a cor de emissão em formato *RGBA* e `intensity` especificando a intensidade luminosa da fonte.

```
material = "emissive"
color = [1.0, 1.0, 1.0, 1.0]
intensity = 15.0
```

Carregamento de Modelos OBJ

O sistema usa a biblioteca *tobj* para carregar modelos OBJ.

```
let (models, _materials) = tobj::load_obj(path, &LoadOptions::default());
```

Durante o carregamento, o sistema extrai posições de vértices, índices de triângulos e coordenadas de textura.

No entanto, o sistema não suporta carregamento de materiais a partir de ficheiros OBJ.

Tonemapping

Para melhorar as cores reproduzidas pelo *pathtracer*, foi implementado um sistema de *tonemapping*. Este sistema, como dito anteriormente, é aplicado no CPU antes da transferência para a textura da GPU.

O sistema suporta vários operadores de *tonemapping* predefinidos, que podem ser alterados na UI em tempo real. Adicionar novos *tonemappers* é simples, basta criar uma nova variante de Tonemapper e definir a função de conversão.

Tonemapping Reinhard

Um *tonemapper* simples que implementámos foi o *Reinhard*. Este *tonemapper* é um dos mais populares e é baseado na fórmula:

$$V_i = \frac{C_i}{1 + C_i}$$

onde C_i é um canal de cor i (RGB) e V_i é o valor final desse canal.

Tonemapping Reinhard com Saturação H&K

Um *tonemapper* que desenvolvemos para melhorar o *Reinhard* é uma variante que incorpora saturação. Este *tonemapper* é baseado na fórmula:

$$R = \frac{L}{1 + L}$$
$$\text{Sat} = \frac{\max(R, G, B) - \min(R, G, B)}{\max(R, G, B)}$$
$$A = R \times (1 + 0.2 \text{ Sat})$$
$$\text{Scale} = \frac{A}{L}$$
$$V_i = C_i \times \text{Scale}$$

onde R , G e B são os canais de cor, L é a luminância do píxel, C_i é um canal de cor i (RGB) e V_i é o valor final desse canal.

Estrutura de Aceleração (BVH) via Embree

Para geometrias complexas, como as malhas que requerem uma grande quantidade de triângulos, fazer o teste de interseção de um raio para cada um deles é demasiado custoso.

Para resolver isto, recorreremos à biblioteca *Embree* da *Intel*, que implementa uma *BVH* para acelerar o teste de interseções para geometrias genéricas.

A *BVH* é uma árvore binária onde cada nó contém um *bounding box* que envolve os objetos que estão contidos nele. Para cada raio, é feito o teste de interseção com o *bounding box* do nó, e se o raio interseccionar o *bounding box*, é feito o teste de interseção com os objetos que estão contidos no nó.

Para construir a *BVH*, é necessário fazer “upload” dos dados para a estrutura de dados do *Embree* e depois chamar a função de construção da *BVH*. Este “upload”, como não suporta todos os tipos de geometrias que implementámos, há um processo de conversão feito por nossa parte.

O *Embree* suporta nativamente triângulos e esferas. Assim sendo, as esferas podem ser cobertas pelo *Embree* sem necessidade de conversão. As malhas de triângulos igualmente, apenas necessitando de uma conversão de tipo de dados para o formato que o *Embree* requer.

Para os *quads*, como não são suportados nativamente, são convertidos em dois triângulos. Com o mesmo raciocínio, para as *boxes*, são decompostas em 6 *quads*, ou seja em 12 triângulos.

Ao fazer o teste de interseção, as coordenadas *UV* não são devolvidas correctamente pelo *Embree*. Assim sendo, temos que fazer a conversão do ponto de interseção para as coordenadas *UV* da geometria original.

Para as esferas, a conversão é feita através do ângulo das coordenadas esféricas. Para as *meshes* de triângulos, a conversão é feita através da interpolação baricêntrica. Para os *quads* e *boxes*, a conversão é feita através da interpolação bilinear, onde no caso dos *boxes* já temos a face que estamos a interseccionar.

A implementação da *BVH* deu-nos ganhos de velocidade extremamente significativos, como se pode ver na tabela abaixo.

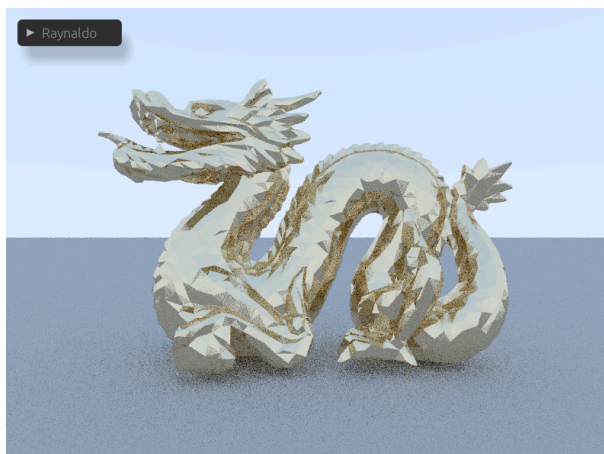


Figura 3: Cena do dragão de *Stanford*

Para uma cena de dragão de *Stanford* (Figura 3), com 5 *SPP* e uma janela 800x600 a correr num *Ryzen 7 7700X* nas cenas *dragon8k.toml* (8000 triângulos) e *dragon80k.toml* (80000 triângulos).

<i>Tracer</i>	dragon8k.toml	dragon80k.toml
<i>Naive</i>	36.5s	355.6s
<i>Embree</i>	0.096s	0.112s

Tabela 1: Tempos de renderização para a cena do dragão

Como podemos ver, com o uso da *BVH*, o tempo de renderização é diminuído drasticamente.

Demonstrações

Apresentamos de seguida algumas demonstrações das cenas desenvolvidas, evidenciando a qualidade e versatilidade do *pathtracer* implementado. Todas as imagens foram renderizadas com 2000 amostras por píxel (*SPP*) para garantir uma grande qualidade visual.

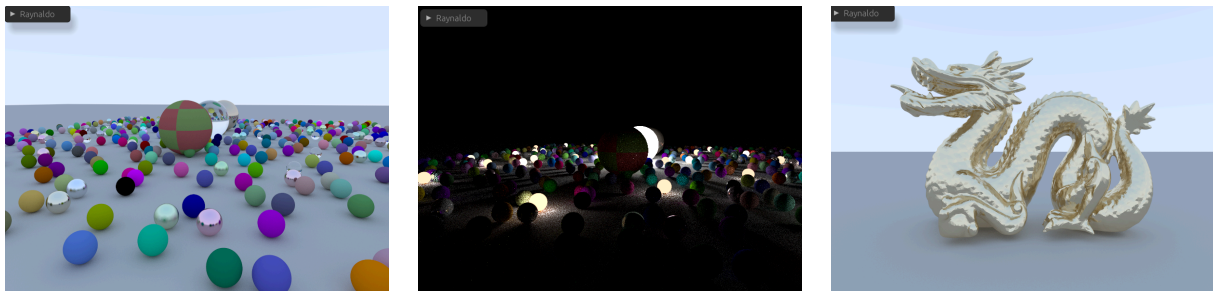


Figura 4: Demonstrações das cenas com mundo aberto

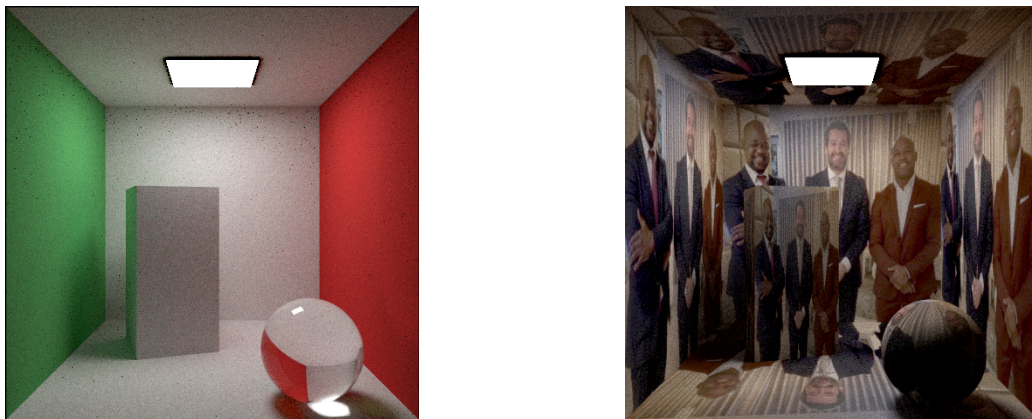


Figura 5: Demonstrações numa *Cornell Box*

Conclusão

Este projeto representou uma grande aprendizagem no desenvolvimento de sistemas de renderização avançados e na implementação de técnicas de *pathtracing*. O trabalho foi bem-sucedido, tendo sido possível implementar todas as funcionalidades propostas com exceção do *Participating Media*, que foi removido do âmbito do projeto. A experiência adquirida na integração de diferentes tecnologias e no desenvolvimento do *pathtracer* mostrou-se extremamente valiosa.

Os resultados obtidos demonstram que os objetivos foram alcançados com sucesso, resultando num *pathtracer* funcional e eficiente.

Para trabalhos futuros, ainda poderíamos ter feito mais, como:

- Suporte completo para carregamento de materiais em ficheiros *OBJ*;
- Redução de ruído para renderizações com poucas amostras;
- Alteração do mundo na *UI*;
- Guardar imagens de *output* do *pathtracer*.